

Randomized Algorithms Problem Set

Instructor: Jing Chen

10 points each problem. Due by Lecture 6.

1. Chapter 13, Problem 1: 3-Coloring.

3-Coloring is a yes/no question, but we can phrase it as an optimization problem as follows.

Suppose we are given a graph $G = (V, E)$, and we want to color each node with one of three colors, even if we aren't necessarily able to give different colors to every pair of adjacent nodes. Rather, we say that an edge (u, v) is satisfied if the colors assigned to u and v are different.

Consider a 3-coloring that maximizes the number of satisfied edges, and let c^* denote this number. Give a polynomial-time algorithm that produces a 3-coloring that satisfies at least $\frac{2}{3}c^*$ edges. If you want, your algorithm can be randomized; in this case, the expected number of edges it satisfies should be at least $\frac{2}{3}c^*$.

2. Chapter 13, Problem 3: Independent Set.

In Section 13.1, we saw a simple distributed protocol to solve a particular contention-resolution problem. Here is another setting in which randomization can help with contention resolution, through the distributed construction of an independent set.

Suppose we have a system with n processes. Certain pairs of processes are in conflict, meaning that they both require access to a shared resource. In a given time interval, the goal is to schedule a large subset S of the processes to run—the rest will remain idle—so that no two conflicting processes are both in the scheduled set S . We'll call such a set S *conflict-free*.

One can picture this process in terms of a graph $G = (V, E)$ with a node representing each process and an edge joining pairs of processes that are in conflict. It is easy to check that a set of processes S is conflict-free if and only if it forms an independent set in G . This suggests that finding a maximum-size conflict-free set S , for an arbitrary conflict G , will be difficult (since the general Independent Set Problem is reducible to this problem). Nevertheless, we can still look for heuristics that find a reasonably large conflict-free set. Moreover, we'd like a simple method for achieving this without centralized control: Each process should communicate with only a small number of other processes and then decide whether or not it should belong to the set S .

We will suppose for purposes of this question that each node has exactly d neighbors in the graph G . (That is, each process is in conflict with exactly d other processes.)

(a) Consider the following simple protocol.

Each process P_i independently picks a random value x_i ; it sets x_i to 1 with probability $\frac{1}{2}$ and sets x_i to 0 with probability $\frac{1}{2}$. It then decides to enter the set S if and only if it chooses the value 1, and each of the processes with which it is in conflict chooses the value 0.

Prove that the set S resulting from the execution of this protocol is conflict-free. Also, give a formula for the expected size of S in terms of n (the number of processes) and d (the number of conflicts per process).

(b) The choice of the probability $\frac{1}{2}$ in the protocol above was fairly arbitrary, and it's not clear that it should give the best system performance. A more general specification of the protocol would replace the probability $\frac{1}{2}$ by a parameter p between 0 and 1, as follows.

Each process P_i independently picks a random value x_i ; it sets x_i to 1 with probability p and sets x_i to 0 with probability $1 - p$. It then decides to enter the set S if and only if it chooses the value 1, and each of the processes with which it is in conflict chooses the value 0.

In terms of the parameters of the graph G , give a value of p so that the expected size of the resulting set S is as large as possible. Give a formula for the expected size of S when p is set to this optimal value.

3. Chapter 13, Problem 4: Peer-to-peer.

A number of peer-to-peer systems on the Internet are based on overlay networks. Rather than using the physical Internet topology as the network on which to perform computation, these systems run protocols by which nodes choose collections of virtual “neighbors” so as to define a higher-level graph whose structure may bear little or no relation to the underlying physical network. Such an overlay network is then used for sharing data and services, and it can be extremely flexible compared with a physical network, which is hard to modify in real time to adapt to changing conditions.

Peer-to-peer networks tend to grow through the arrival of new participants, who join by linking into the existing structure. This growth process has an intrinsic effect on the characteristics of the overall network. Recently, people have investigated simple abstract models for network growth that might provide insight into the way such processes behave, at a qualitative level, in real networks.

Here's a simple example of such a model. The system begins with a single node v_1 . Nodes then join one at a time; as each node joins, it executes a protocol whereby it forms a directed link to a single other node chosen uniformly at random from those already in the system. More concretely, if the system already contains nodes $v_1, v_2, \dots, v_{k-1}$ and node v_k wishes to join, it randomly selects one of $v_1, v_2, \dots, v_{k-1}$ and links to this node.

Suppose we run this process until we have a system consisting of nodes $v_1, v_2, \dots, v_n$; the random process described above will produce a directed network in which each node other than v_1 has exactly one outgoing edge. On the other hand, a node may have multiple incoming links, or none at all. The incoming links to a node v_j reflect all the other nodes whose access into the system is via v_j ; so if v_j has many incoming links, this can place a large load on it. To keep the system load-balanced, then, we'd like all nodes to have a roughly comparable number of incoming links. That's unlikely to happen here, however, since nodes that join earlier in the process are likely to have more incoming links than nodes that join later. Let's try to quantify this imbalance as follows.

(a) Given the random process described above, what is the expected number of incoming links to node v_j in the resulting network? Give an exact formula in terms of n and j , and also try to express this quantity asymptotically (via an expression without large summations) using $\Theta(\cdot)$ notation.

(b) Part (a) makes precise a sense in which the nodes that arrive early carry an "unfair" share of the connections in the network. Another way to quantify the imbalance is to observe that, in a run of this random process, we expect many nodes to end up with no incoming links.

Give a formula for the expected number of nodes with no incoming links in a network grown randomly according to this model.

4. Chapter 13, Problem 7: MAX SAT.

In Section 13.4, we designed an approximation algorithm to within a factor of $7/8$ for the MAX 3-SAT Problem, where we assumed that each clause has terms associated with three different variables. In this problem, we will consider the analogous MAX SAT Problem: Given a set of clauses $C_1, \dots, C_k$ over a set of variables $X = \{x_1, \dots, x_n\}$, find a truth assignment satisfying as many of the clauses as possible. Each clause has at least one term in it, and all the variables in a single clause are distinct, but otherwise we do not make any assumptions on the length of the clauses: There may be clauses that have a lot of variables, and others may have just a single variable.

(a) First consider the randomized approximation algorithm we used for MAX 3-SAT, setting each variable independently to true or false with probability $1/2$ each. Show that the expected number of clauses satisfied by this random assignment is at least $k/2$, that is, at least half of the clauses are satisfied in expectation. Give an example to show that there are MAX SAT instances such that no assignment satisfies more than half of the

clauses.

(b) If we have a clause that consists only of a single term (e.g., a clause consisting just of x_1 , or just of $\overline{x_2}$), then there is only a single way to satisfy it: We need to set the corresponding variable in the appropriate way. If we have two clauses such that one consists of just the term x_i , and the other consists of just the negated term $\overline{x_i}$, then this is a pretty direct contradiction.

Assume that our instance has no such pair of “conflicting clauses”; that is, for no variable x_i do we have both a clause $C = \{x_i\}$ and a clause $C' = \{\overline{x_i}\}$. Modify the randomized procedure above to improve the approximation factor from $1/2$ to at least $.6$. That is, change the algorithm so that the expected number of clauses satisfied by the process is at least $.6k$.

(c) Give a randomized polynomial-time algorithm for the general MAX SAT Problem, so that the expected number of clauses satisfied by the algorithm is at least a $.6$ fraction of the maximum possible.

(Note that, by the example in part (a), there are instances where one cannot satisfy more than $k/2$ clauses; the point here is that we’d still like an efficient algorithm that, in expectation, can satisfy a $.6$ fraction of *the maximum that can be satisfied by an optimal assignment*.)

5. Chapter 13, Problem 9: One-pass Auction.

Suppose you’re designing strategies for selling items on a popular auction Web site. Unlike other auction sites, this one uses a *one-pass auction*, in which each bid must be immediately (and irrevocably) accepted or refused. Specifically, the site works as follows.

- First a seller puts up an item for sale.
- Then buyers appear in sequence.
- When buyer i appears, he or she makes a bid $b_i > 0$.
- The seller must decide immediately whether to accept the bid or not. If the seller accepts the bid, the item is sold and all future buyers are turned away. If the seller rejects the bid, buyer i departs and the bid is withdrawn; and only then does the seller see any future buyers.

Suppose an item is offered for sale, and there are n buyers, each with a distinct bid. Suppose further that the buyers appear in a random order, and that the seller knows the number n of buyers. We’d like to design a strategy whereby the seller has a reasonable chance of accepting the highest of the n bids. By a *strategy*, we mean a rule by which the seller decides whether to accept each presented bid, based only on the value of n and the sequence of bids seen so far.

For example, the seller could always accept the first bid presented. This results in the seller accepting the highest of the n bids with probability only $1/n$, since it requires the highest bid to be the first one presented.

Give a strategy under which the seller accepts the highest of the n bids with probability at least $1/4$, regardless of the value of n . (For simplicity, you may assume that n is an even number.) Prove that your strategy achieves this probabilistic guarantee.

6. Chapter 13, Problem 15: Approximate Median.

Suppose you are presented with a very large set S of real numbers, and you'd like to approximate the median of these numbers by sampling. You may assume all the numbers in S are distinct. Let $n = |S|$; we will say that a number x is an ϵ -approximate median of S if at least $(\frac{1}{2} - \epsilon)n$ numbers in S are less than x , and at least $(\frac{1}{2} - \epsilon)n$ numbers in S are greater than x .

Consider an algorithm that works as follows. You select a subset $S' \subseteq S$ uniformly at random, compute the median of S' , and return this as an approximate median of S . Show that there is an absolute constant c , independent of n , so that if you apply this algorithm with a sample S' of size c , then with probability at least .99, the number returned will be a (.05)-approximate median of S . (You may consider either the version of the algorithm that constructs S' by sampling with replacement, so that an element of S can be selected multiple times, or one without replacement.)

7. Chapter 13, Problem 8: Dense Subgraph.

Let $G = (V, E)$ be an undirected graph with n nodes and m edges. For a subset $X \subseteq V$, we use $G[X]$ to denote the subgraph *induced* on X —that is, the graph whose node set is X and whose edge set consists of all edges of G for which both ends lie in X .

We are given a natural number $k \leq n$ and are interested in finding a set of k nodes that induces a “dense” subgraph of G ; we'll phrase this concretely as follows. Give a polynomial-time algorithm that produces, for a given natural number $k \leq n$, a set $X \subseteq V$ of k nodes with the property that the induced subgraph $G[X]$ has at least $\frac{mk(k-1)}{n(n-1)}$ edges.

You may give either (a) a deterministic algorithm, or (b) a randomized algorithm that has an expected running time that is polynomial, and that only outputs correct answers.

8. Chapter 13, Problem 6: Betweenness.

One of the (many) hard problems that arises in genome mapping can be formulated in the following abstract way. We are given a set of n markers $\{\mu_1, \dots, \mu_n\}$ —these are positions on a chromosome that we are trying to map—and our goal is to output a linear ordering of these markers. The output should be consistent with a set of k constraints,

each specified by a triple (μ_i, μ_j, μ_k) , requiring that μ_j lie *between* μ_i and μ_k in the total ordering that we produce. (Note that this constraint does not specify which of μ_i or μ_k should come first in the ordering, only that μ_j should come between them.)

Now it is not always possible to satisfy all constraints simultaneously, so we wish to produce an ordering that satisfies as many as possible. Unfortunately, deciding whether there is an ordering that satisfies at least k' of the k constraints is an NP-complete problem (you don't have to prove this.)

Give a constant $\alpha > 0$ (independent of n) and an algorithm with the following property. If it is possible to satisfy k^* of the constraints, then the algorithm produces an ordering of markers satisfying at least αk^* of the constraints. Your algorithm may be randomized; in this case it should produce an ordering for which the *expected* number of satisfied constraints is at least αk^* .

9. Chapter13, Problem 12: Contraction.

Consider the following analogue of Karger's algorithm for finding minimum s - t cuts. We will contract edges iteratively using the following randomized procedure. In a given iteration, let s and t denote the possibly contracted nodes that contain the original nodes s and t , respectively. To make sure that s and t do not get contracted, at each iteration we delete any edges connecting s and t and select a random edge to contract among the remaining edges. Give an example to show that the probability that this method finds a minimum s - t cut can be exponentially small.

10. Chapter13, Problem 13: Balls-and-Bins.

Consider a balls-and-bins experiment with $2n$ balls but only two bins. As usual, each ball independently selects one of the two bins, both bins equally likely. The expected number of balls in each bin is n . In this problem, we explore the question of how big their difference is likely to be. Let X_1 and X_2 denote the number of balls in the two bins, respectively. (X_1 and X_2 are random variables.) Prove that for any $\varepsilon > 0$ there is a constant $c > 0$ such that the probability $\Pr[X_1 - X_2 > c\sqrt{n}] \leq \varepsilon$.

11. Chapter13, Problem 11: Load Balancing.

Load balancing algorithms for parallel or distributed systems seek to spread out collections of computing jobs over multiple machines. In this way, no one machine becomes a "hot spot". If some kind of central coordination is possible, then the load can potentially be spread out almost perfectly. But what if the jobs are coming from diverse sources that can't coordinate? As we saw in Section 13.10, one option is to assign them to machines at random and hope that this randomization will work to prevent imbalances. Clearly, this won't generally work as well as a perfectly centralized solution, but it can be quite effective. Here we try analyzing some variations and extensions on the simple

load balancing heuristic we considered in Section 13.10.

Suppose you have k machines, and k jobs show up for processing. Each job is assigned to one of the k machines independently at random (with each machine equally likely).

(a) Let $N(k)$ be the expected number of machines that do not receive any jobs, so that $N(k)/k$ is the expected fraction of machines with nothing to do. What is the value of the limit $\lim_{k \rightarrow \infty} N(k)/k$? Give a proof of your answer.

(b) Suppose that machines are not able to queue up excess jobs, so if the random assignment of jobs to machines sends more than one job to a machine M , then M will do the first of the jobs it receives and reject the rest. Let $R(k)$ be the expected number of rejected jobs; so $R(k)/k$ is the expected fraction of rejected jobs. What is $\lim_{k \rightarrow \infty} R(k)/k$? Give a proof of your answer.

(c) Now assume that machines have slightly larger buffers; each machine M will do the first two jobs it receives, and reject any additional jobs. Let $R_2(k)$ denote the expected number of rejected jobs under this rule. What is $\lim_{k \rightarrow \infty} R_2(k)/k$? Give a proof of your answer.

12. Chapter 13, Problem 14: Perfectly Balanced.

Some people designing parallel physical simulations come to you with the following problem. They have a set P of k *basic processes* and want to assign each process to run on one of two machines, M_1 and M_2 . They are then going to run a sequence of n *jobs*, $J_1, \dots, J_n$. Each job J_i is represented by a set $P_i \subseteq P$ of exactly $2n$ basic processes which must be running (each on its assigned machine) while the job is processed. An assignment of basic processes to machines will be called *perfectly balanced* if, for each job J_i , exactly n of the basic processes associated with J_i have been assigned to each of the two machines. An assignment of basic processes to machines will be called *nearly balanced* if, for each job J_i , no more than $\frac{4}{3}n$ of the basic processes associated with J_i have been assigned to the same machine.

(a) Show that for arbitrarily large values of n , there exist sequences of jobs $J_1, \dots, J_n$ for which no perfectly balanced assignment exists.

(b) Suppose that $n \geq 200$. Give an algorithm that takes an arbitrary sequence of jobs $J_1, \dots, J_n$ and produces a nearly balanced assignment of basic processes to machines. Your algorithm may be randomized, in which case its expected running time should be polynomial, and it should always produce the correct answer.